Saga

eason: 4 · Enterprise · Category: Enterprise · Level: Intermediate · Build with Waqa

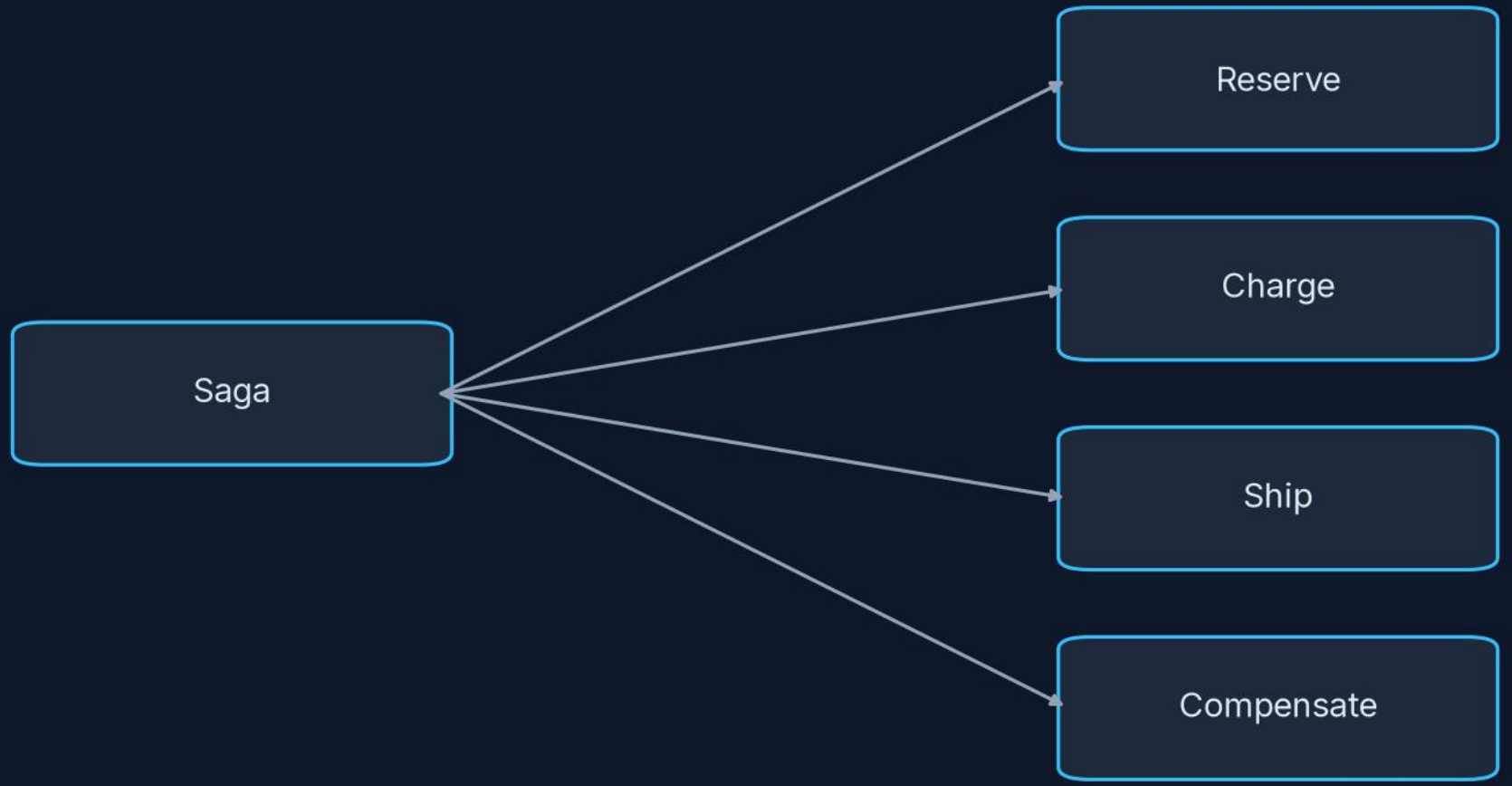
Today's Hook

- Today: Saga as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Saga structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Saga
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/enterprise/saga/OrderSaga.java
package com.buildwithwaqar.patterns.enterprise.saga;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Supplier;

/** Minimal choreography-style saga with compensations. */
public final class OrderSaga {
    private final List<Runnable> compensations = new ArrayList<>();
    private final List<String> log = new ArrayList<>();

    public void step(String name, Runnable action, Runnable compensate) {
        action.run();
        log.add("ok:" + name);
        compensations.addFirst(compensate);
    }

    public void run(Supplier<Boolean> paymentOk) {
        try {
            step("reserve", () -> log.add("reserved"), () -> log.add("release"));
            step("charge", () -> {
                if (!paymentOk.get()) throw new IllegalStateException("pay");
                log.add("charged");
            }, () -> log.add("refund"));
            step("ship", () -> log.add("shipped"), () -> log.add("recall"));
        } catch (RuntimeException e) {
            for (var c : compensations) c.run();
            log.add("aborted");
        }
```

Implementation (2)

```
    }

    List<String> log() { return List.copyOf(log); }
}
```

Demo.main

```java
package com.buildwithwaqar.patterns.enterprise.saga;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Saga walkthrough ===");
    var ok = new OrderSaga(); ok.run(() -> true);
    System.out.println("success=" + ok.log());
    var bad = new OrderSaga(); bad.run(() -> false);
    System.out.println("compensated=" + bad.log());
  }
}
```

Walkthrough Output

```
=== Saga walkthrough ===
success=[reserved, ok:reserve, charged, ok:charge, shipped, ok:ship]
compensated=[reserved, ok:reserve, release, aborted]
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Saga
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Saga → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.